Software Architecture Document (SAD)

Student Task Planner

Name:Yade Başkan

Course:Software Architecture

Instructor:Bahman Arasteh

Date:04.05.2026

1. Introduction

1.1 Purpose

This document describes the architecture of the **Student Task Planner** application.
This document provides a comprehensive architectural overview of the Student Task Planner application. It details the design decisions, structural patterns, and component interactions used to build the system.

1.2 Scope

The Student Task Planner is a web-based productivity tool designed for students to manage academic and personal responsibilities. It features secure user authentication, task lifecycle management, and data visualization through integrated charts and calendars.
The Student Task Planner is a web-based application that allows users to:

- Register and login securely
- Create, update, and delete tasks
- Track task status (todo / done)
- Assign deadlines to tasks
- Visualize tasks using charts
- View tasks in a calendar interface
- Filter tasks by status

1.3 Architectural Representation (3-Tier Layered Architecture)

The system follows a **Client-Server Architecture**:

- **Frontend:** React (Vite)
- **Backend:** Node.js (Express)

- **Database:** SQLite

The system uses REST API communication between frontend and backend.

1.4 Technical Platform

- React.js (Frontend)
- Node.js + Express (Backend)
- SQLite (Database)
- Chart.js (Visualization)
- React Calendar (Calendar UI)
- JWT (Authentication)

1.5 Transactions

The system handles the following operations:

- Task creation
- Task update (status toggle)
- Task deletion

All operations are performed through HTTP requests and stored in the database.

1.6 Quality Attributes

Security

- JWT-based authentication
- User-specific data access (user_id)

Performance

- Fast frontend rendering using React
- Lightweight backend using Express

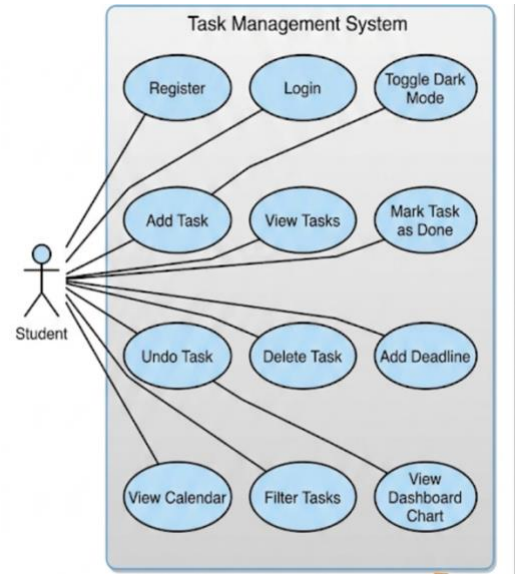
Reliability

- Persistent storage using SQLite

2. Use-Case View

2.1 Main Use Cases

- **User Management:** Secure Registration, Login, and Session handling.
- **Task Management:** Create, Read, Update (Toggle Status), and Delete (CRUD) operations.
- **Visualization:** Interactive Dashboard with Chart.js and a dedicated Calendar UI.
- **Personalization:** Accessibility features including Dark Mode toggle.



2.2 Use-Case Realization Example

Add Task Flow:

1. User enters task title and deadline
2. Frontend sends POST request
3. Backend processes request
4. Task is stored in database
5. Frontend refreshes task list

Error Flow: Unauthorized Access

1. User attempts to access or modify tasks without a valid session.
2. The Logic Layer intercepts the request and verifies the **JWT (JSON Web Token)**.
3. If the token is missing or invalid, the system returns a **401 Unauthorized** status code.
4. The Frontend redirects the user to the login page with a descriptive error message.

3. Logical View

3.1 Overview

The system is divided into three layers:

1. Presentation Layer (Frontend)
2. Application Layer (Backend)
3. Database Layer(Database)

REST API Design

The backend exposes a RESTful interface for seamless communication between the frontend and the database:

• POST /auth/register : Creates a new user profile.

• POST /auth/login : Authenticates credentials and issues a JWT.

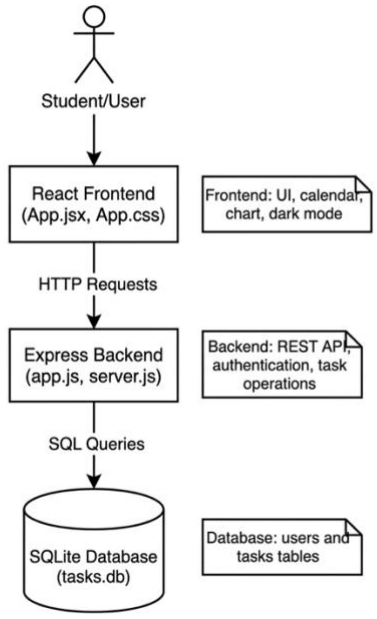
• GET /tasks : Fetches tasks filtered by the authenticated user's ID.

• POST /tasks : Persists a new task entry.

• PUT /tasks/:id : Updates existing task metadata (title, deadline, or status).

• DELETE /tasks/:id : Permanently removes a task record.

2. Logical Architecture Diagram



3.2Components

Frontend: App.jsx, UI components (task list, calendar, chart)

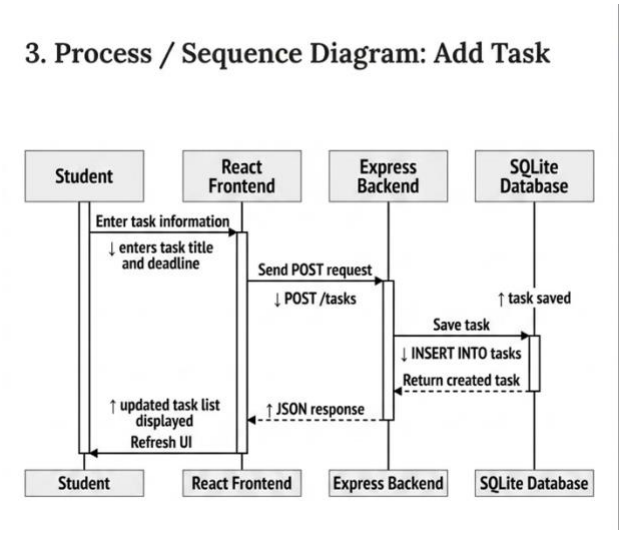
Backend: Express server, REST API endpoints

Database: SQLite database storing users and tasks

4. Process View

The system follows a request-response model:

1. User interacts with frontend
2. Frontend sends HTTP request
3. Backend processes request
4. Database is updated/read
5. Response returned to frontend



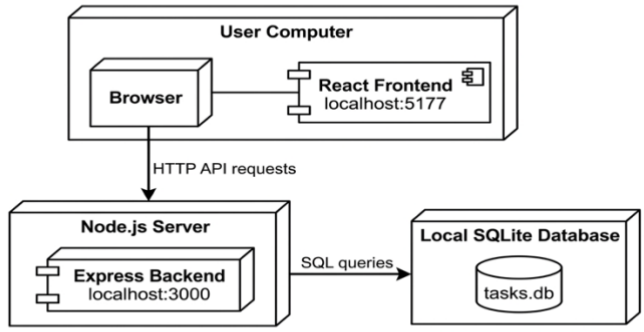
5. Deployment View

The system runs locally:

- Frontend → localhost:5177
- Backend → localhost:3000
- Database → SQLite (tasks.db)

4. Deployment Diagram

Kutular:



Oklar:

- Browser → Node.js Server: HTTP API requests
- Node.js Server → SQLite Database: SQL queries

6. Implementation View

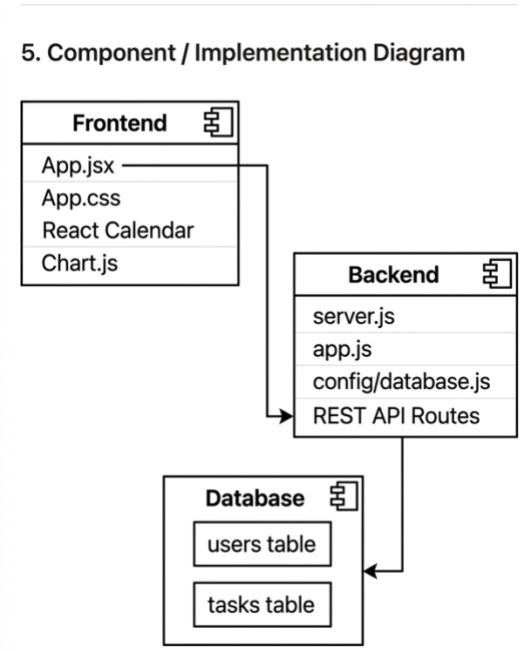
6.1 Overview

The system is implemented using JavaScript technologies:

- React (Frontend)
- Node.js (Backend)

6.2 Layers

- **Presentation Layer:** Developed with React.js (Vite) for a responsive and dynamic User Interface.
- **Logic Layer:** Powered by Node.js and Express.js to handle business logic and API routing.
- **Data Layer:** Utilizes SQLite for lightweight, persistent, and relational data storage.



7. Data View (Database Design)

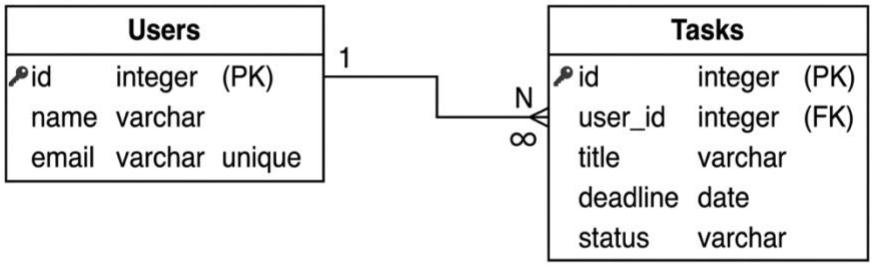
Users Table

- id (Primary Key)
- name
- email
- password

Tasks Table

- id (Primary Key)
- title
- status
- deadline
- user_id (Foreign Key)

Task Management System Database Schema



8. Security

- JWT token authentication
- User-based data filtering
- Secure password hashing (bcrypt)

9. Future Improvements

- Mobile application version
- Push notifications for deadlines
- Task sharing between users
- Drag & drop task management
- Cloud deployment (AWS / Vercel)

10. References

- React Documentation
- Node.js Documentation
- Express.js
- SQLite
- Chart.js